

ONLINE ASSIGNMENT 1

Ôn tập Lập trình Python cơ bản và thao tác trên cấu trúc dữ liệu

1. Đề bài

Xây dựng chương trình quản lý điểm sinh viên bằng hàm, cấu trúc dữ liệu và một số kỹ thuật xử lý dữ liệu nâng cao trong Python.

Mục tiêu:

Sinh viên vận dụng kiến thức về hàm, list, dict, string và cấu trúc điều khiển để xây dựng chương trình quản lý điểm sinh viên. Ngoài các yêu cầu cơ bản, sinh viên cần sử dụng thêm một số kỹ thuật nâng cao ở mức phù hợp như list comprehension, dictionary comprehension, hàm lambda, hàm sorted() với tham số key, và xử lý ngoại lệ cơ bản.

2. Nội dung kiến thức nâng cao được tích hợp

Sinh viên cần tìm hiểu và vận dụng các nội dung sau:

2.1. List comprehension

List comprehension giúp tạo danh sách mới từ danh sách có sẵn một cách ngắn gọn.

Ví dụ:

```
scores = [4.5, 7.0, 8.5, 6.0, 9.0]
passed_scores = [score for score in scores if score >= 5.5]
print(passed_scores)
```

Kết quả:

```
[7.0, 8.5, 6.0, 9.0]
```

2.2. Dictionary comprehension

Dictionary comprehension giúp tạo từ điển mới từ dữ liệu có sẵn.

Ví dụ:

```
students = [
    {"id": "SV01", "name": "An", "score": 8.0},
    {"id": "SV02", "name": "Binh", "score": 6.5},
    {"id": "SV03", "name": "Chi", "score": 9.0}
]
```

```
student_scores = {student["id"]: student["score"] for student in
students}

print(student_scores)
```

Kết quả:

```
{'SV01': 8.0, 'SV02': 6.5, 'SV03': 9.0}
```

2.3. Sắp xếp dữ liệu với sorted() và lambda

Có thể sắp xếp danh sách dictionary theo một khóa cụ thể.

Ví dụ:

```
students = [
    {"id": "SV01", "name": "An", "final_score": 8.0},
    {"id": "SV02", "name": "Binh", "final_score": 6.5},
    {"id": "SV03", "name": "Chi", "final_score": 9.0}
]

students_sorted = sorted(
    students,
    key=lambda student: student["final_score"],
    reverse=True
)

for student in students_sorted:
    print(student["id"], student["name"], student["final_score"])
```

2.4. Xử lý ngoại lệ khi nhập dữ liệu

Xử lý ngoại lệ giúp chương trình tránh bị dừng đột ngột khi người dùng nhập sai dữ liệu.

Ví dụ:

```
try:
```

```

score = float(input("Nhập điểm: "))

if score < 0 or score > 10:
    print("Điểm phải nằm trong khoảng từ 0 đến 10")
else:
    print("Điểm hợp lệ:", score)

except ValueError:
    print("Dữ liệu nhập vào không phải là số")

```

3. Hướng dẫn

Sinh viên xây dựng chương trình quản lý điểm sinh viên. Mỗi sinh viên gồm:

- Mã sinh viên
- Họ tên
- Điểm chuyên cần
- Điểm kiểm tra
- Điểm thi
- Điểm tổng kết
- Xếp loại

Công thức tính điểm tổng kết:

$diem_tong_ket = 0.1 * diem_chuyen_can + 0.3 * diem_kiem_tra + 0.6 * diem_thi$

Quy tắc xếp loại:

Điểm tổng kết	Xếp loại
Từ 8.5 đến 10	Giỏi
Từ 7.0 đến dưới 8.5	Khá
Từ 5.5 đến dưới 7.0	Trung bình
Dưới 5.5	Yếu

4. Yêu cầu thực hiện

Sinh viên thực hiện các yêu cầu sau:

1. Tạo danh sách sinh viên bằng list chứa các dictionary.
2. Viết hàm tính điểm tổng kết cho từng sinh viên.
3. Viết hàm xếp loại sinh viên.
4. Cập nhật điểm tổng kết và xếp loại vào từng dictionary sinh viên.
5. In danh sách sinh viên kèm điểm tổng kết và xếp loại.
6. Sử dụng `sorted()` và `lambda` để sắp xếp danh sách sinh viên theo điểm tổng kết giảm dần.
7. Sử dụng list comprehension để tạo danh sách các sinh viên đạt yêu cầu, với điều kiện điểm tổng kết ≥ 5.5 .
8. Sử dụng dictionary comprehension để tạo từ điển có khóa là mã sinh viên và giá trị là điểm tổng kết.
9. Tìm sinh viên có điểm tổng kết cao nhất.
10. Thống kê số lượng sinh viên theo từng xếp loại.
11. Có xử lý ngoại lệ cơ bản khi nhập điểm từ bàn phím nếu chương trình có chức năng nhập dữ liệu.

5. Yêu cầu sản phẩm

Sinh viên nộp lên LMS:

1. File mã nguồn: `Bai10_MaSV_HoTen.py`
2. Ghi chú ngắn mô tả:
 - Cấu trúc dữ liệu đã sử dụng.
 - Hàm chính trong chương trình.
 - Nội dung nâng cao đã áp dụng: list comprehension, dictionary comprehension, `lambda`, `sorted()` hoặc xử lý ngoại lệ.